TASK-006 - Passo Relâmpago

Spirit Gate | 18/10/2026 | Java 17 e Maven

Estado: implementação e participação do usuário validadas. O teste reservado de direção zero foi escrito pelo usuário e passou na suíte completa. A conferência manual interativa da sensação de jogo continua recomendada.

Resultado implementado

Shift captura a direção até o ponto do mouse e publica SkillCastCommand com ID lightning_step. Uma ativação custa 35 de vigor e inicia cooldown de 4 segundos. O jogador se desloca progressivamente a 600 px/s durante 0,25 s, com orçamento máximo de 150 px antes das colisões. Não há teleporte no lançamento, dano, invulnerabilidade ou efeito de status.

O ponto do mouse define somente a direção, fixada no início. Um cursor próximo ainda solicita os 150 px nessa direção; colisões podem encurtar o percurso. Essa decisão foi apresentada antes da implementação. O alvo precisa estar dentro da arena e ser finito. A validação de direção zero está implementada e seu teste foi escrito pelo usuário.

Preparação e arquitetura

Foram inspecionados MovementService, CollisionService, StaminaService, SkillCastCommand, DebugCommandBuffer, DebugRuntime, DebugWorld, DebugLauncher, DebugSnapshot e DebugPanel. O estado inicial passou em mvn clean test -q com 118 testes.

A proposta apresentada criou somente um serviço de habilidade específico e reutilizou contrato, fila, consumo de vigor e física contínua. A movimentação e a Criostase mantêm seus serviços existentes. Não foram adicionadas dependências, exclusões, renomeações, rede ou alterações em banco.

LightningStepService é o estado de uma ativação por jogador: direção, duração restante e cooldown. Recebe MovementService, StaminaService e CollisionService. cast valida comando e geometria inicial antes de gastar. update avança cooldown e deslocamento, retornando um Step imutável com as posições antes/depois. O serviço não conhece Swing, snapshots ou o efeito visual.

O construtor recebe a mesma instância de detecção usada na movimentação. Assim, cast consulta origem livre antes de consumir; não deixa para descobrir uma sobreposição na primeira varredura. Cada recusa mantém vigor e cooldown. Morte durante a ação interrompe o movimento seguinte, sem conceder proteção ou restaurar o recurso consumido.

Duração, colisões e movimento normal

O serviço usa min(dt, tempo restante) e consome esse tempo mesmo quando bloqueado. A 30 Hz, são sete passos solicitados de 20 px e um último de 10 px. Não há compensação da distância retida na parede em ticks posteriores. A precisão geométrica é float; verificações de distância usam tolerâncias numéricas pequenas nos testes.

O deslocamento passa exclusivamente por MovementService.moveContinuous, usando Space existente. A varredura mantém a prioridade X antes de Y e permite deslizar no eixo livre, parando no contato com paredes, mobs e limites. O índice espacial continua sendo atualizado pelo CollisionService. A assistência lenta de contorno usada para caminhada não é aplicada ao dash; a física contínua existente foi mantida intacta.

Na arena, o serviço é atualizado no slot de movimento. Quando havia dash ativo no começo do tick, WASD não é executado, inclusive no tick fracionário de conclusão. O movimento normal retorna no tick seguinte, sem aplicar sobras de WASD ao orçamento da habilidade.

O lançamento ocorre no slot anterior de comandos de skill, depois do movimento e ataque daquele tick. Ele inicia o estado sem mover; o primeiro avanço ocorre no próximo tick, com latência de até um passo de simulação (cerca de 33 ms). Assim, mantém-se a ordem anterior da Criostase e do ataque. Se havia WASD no tick de lançamento, ele ocorreu antes de aceitar a habilidade; nenhum tick soma movimento de dash com movimento WASD.

Os efeitos existentes avançam antes dos pulsos da Criostase, como antes. Ataque básico e IA continuam ativos durante o dash. A possibilidade de receber dano foi verificada tanto pelo CombatService quanto pela IA real da arena. Não foram modificadas fórmulas, afinidades, congelamento, pulsos ou parâmetros da Criostase.

Fila e controles

DebugCommandBuffer e DebugRuntime não precisaram ser alterados: a lista de SkillCastCommand já é imutável e preserva a ordem FIFO. DebugWorld despacha lightning_step e cryostasis em um único laço, respeitando o saldo de vigor compartilhado. Se só houver recurso para uma das habilidades, a primeira intenção elegível vence; a recusa da segunda não inicia seu cooldown.

DebugLauncher registra press_SHIFT e release_SHIFT. O conjunto de teclas mantidas publica uma intenção somente na nova pressão; autorepeat e Shift mantido não repetem. Toques completos entre ticks chegam pela fila. A captura do mouse acontece no instante da pressão.

Shift é um modificador do Swing, então os bindings também incluem combinações com SHIFT_DOWN_MASK. Q, WASD, Espaço, E, H e R continuam respondendo quando Shift está mantido. O teste de bindings usa um KeyEvent real com esse modificador.

R cria um mundo novo, eliminando dash, rastros e cooldowns. Comandos anteriores ao reinício são descartados pela fila existente. O conjunto de teclas no EDT não é limpo por R; Shift mantido não cria outra intenção até ser solto e pressionado de novo.

Rastro e snapshot

O mundo publica lightningStepActive, lightningStepRemaining, lightningStepCooldown e uma cópia imutável de LightningTrailView. Os segmentos vêm apenas de deslocamentos reais. Cada passo pode produzir os dois trechos X/Y, representando o caminho da física sem atravessar visualmente uma quina. Cada segmento vive por 0,18 s e desvanece mesmo depois

do dash terminar.

DebugPanel lê os segmentos e desenha um traço amarelo em zigue-zague, com brilho e opacidade baseada no tempo publicado. Não move entidades, não atualiza timers e não executa regras. O HUD mostra Shift, custo e cooldown, inclusive durante o deslocamento, preservando os controles anteriores.

Validação executada

Comando final, após a contribuição do usuário: mvn clean test na pasta server. Resultado real: BUILD SUCCESS, compilação release 17, 147 testes, zero falhas, zero erros e zero ignorados. São 118 testes anteriores, 28 testes implementados pelo agente e o teste de direção zero escrito pelo usuário. A entrega anterior tinha 146 testes e aguardava essa participação. Resumo por classe em TASK-006-testes.txt.

LightningStepServiceTest acrescenta 19 testes: consumo único, ausência de teleporte no cast, progresso, último tick fracionário, duração, passo grande limitado, diagonal, mouse próximo, cooldown, vigor insuficiente/exato, morto, morte durante a ação, entradas inválidas, geometria inválida, dt inválido/zero, paredes finas, deslizamento X/Y, limites, mobs sem dano/status e recebimento de dano pelo CombatService.

LightningStepIntegrationTest acrescenta 9 testes: ativação/publicação de rastro, duração sem soma WASD, cooldown e ataque, ordem Q/Shift com vigor limitado, dano pela IA, rastro imutável/expiração/captura, bindings Shift, toque rápido com reinício sem repetição e descarte de dash pendente.

Os 118 testes anteriores e os três baselines de MovementReplayTest foram preservados. Nenhum teste existente foi removido ou reescrito. Os campos novos ficam fora da projeção histórica já existente. O teste do usuário foi revisado sem alteração de código e está incluído na contagem final de 147.

Captura: TASK-006-passo-relampago.png, produzida pelo teste e inspecionada. Mostra o jogador após cinco passos, rastro amarelo, vigor em regeneração e cooldown no HUD. Não houve sessão interativa com JFrame; a imagem é uma renderização real do snapshot de jogo, e os bindings são testados no EDT com o runtime real.

Participação do usuário - teste validado

Arquivo criado pelo usuário e validado:
server/src/test/java/com/projetok/domain/skill/LightningStepZeroDirectionTest.java

O teste usa o pacote com.projetok.domain.skill, JUnit 5 e as fixtures reais indicadas abaixo. O jogador está em (100,100), e o comando usa exatamente o mesmo ponto. As asserções verificam ZERO_DIRECTION, vigor preservado, cooldown zerado, posição preservada e dash inativo. Nenhuma correção obrigatória foi identificada. O agente não escreveu nem modificou esse teste.

Construtor real:
LightningStepService(MovementService movement, StaminaService stamina, CollisionService

collisions)

Métodos reais:
- Status cast(Player player, SkillCastCommand command, float radius, CollisionService.Space space).
- Step update(Player player, float dt, float radius, CollisionService.Space space).
- boolean active(); float remaining(); float cooldown(); Position direction().
- SkillCastCommand(String skillId, float targetX, float targetY).
- Player.getPosition(); Player.require(StaminaComponent.class).current().

As fixtures existentes ficam em LightningStepTestFixtures.java, no mesmo pacote de testes:
- player(float x, float y): PlayerFactory real; humano com 100 HP e 100 vigor.
- emptySpace(): CollisionService.Space de 960 por 640, sem obstáculos ou corpos.
- service(): novo serviço, com MovementService e CollisionService compartilhados.
- RADIUS: 18f, o mesmo raio do jogador na arena.

São acessíveis diretamente por testes no mesmo pacote. Os resultados possíveis estão no enum LightningStepService.Status. A suíte completa foi executada novamente após o usuário salvar seu teste, e esta entrega foi atualizada. A participação reservada está concluída e validada.

Arquivos novos e alterados

Novos no domínio:
- server/src/main/java/com/projetok/domain/skill/LightningStepService.java.

Alterados no protótipo, sob server/src/main/java/com/projetok/prototype/:
- DebugWorld.java: instância do serviço, despacho FIFO, substituição de WASD e rastros.
- DebugLauncher.java: Shift e aliases de teclas com modificador.
- DebugSnapshot.java: estado por valor e lista imutável de rastros.
- DebugPanel.java: traço elétrico e indicador no HUD.

Novos testes e fixtures, sob server/src/test/java/com/projetok/:
- domain/skill/LightningStepServiceTest.java.
- domain/skill/LightningStepZeroDirectionTest.java (autoria do usuário).
- domain/skill/LightningStepTestFixtures.java.
- prototype/LightningStepIntegrationTest.java.

Documentação e ferramenta:
- README-PROTOTIPO.md e docs/HISTORICO-TECNICO.md.
- docs/TASK-006.md, docs/TASK-006-testes.txt e docs/TASK-006.diff.
- docs/Spirit_Gate_TASK-006.pdf e docs/TASK-006-passo-relampago.png.
- tools/gerar_pdf_task006.py.

DebugCommandBuffer, DebugRuntime, SkillCastCommand, CombatService, CryostasisService, congelamento, recursos e serviços de movimento/colisão permaneceram inalterados. Nenhuma dependência, banco, multiplayer ou habilidade adicional foi implementada.

Reprodução e limites

Na pasta server: mvn clean test. Na pasta Projeto-K: python -B tools/gerar_pdf_task006.py regenera o PDF. Para validar manualmente, iniciar rodar-prototipo.cmd, apontar dentro da arena e pressionar Shift, testar parede/mob, manter WASD, observar retorno da caminhada e usar R mantendo Shift.

A instância do serviço pertence a um jogador e à thread autoritativa. A duração progride mesmo bloqueada; não há busca de caminho. O rastro é apenas visual. A morte interrompe o dash na atualização seguinte; o dano pode continuar sendo aplicado normalmente.

A pasta não é um checkout Git acessível; não houve branch ou merge. O diff é relativo ao estado anterior à TASK-006, sem target, segredos, binários temporários ou o próprio diff recursivamente, e inclui a contribuição do usuário com sua autoria registrada. PDF e captura são artefatos finais separados. A implementação e a participação reservada estão validadas; uma sessão manual continua recomendada para conferir a sensação de jogo.